

Week 1

Exercise 2: Implementing the Factory Method Pattern

Main.java

```
package org.example;

//TIP To <b>Run</b> code, press <shortcut actionId="Run"/> or
// click the <icon src="AllIcons.Actions.Execute"/> icon in the gutter.
public class Main {
    public static void main(String[] args) {
        //TIP Press <shortcut actionId="ShowIntentionActions"/> with your
        // caret at the highlighted text
        // to see how IntelliJ IDEA suggests fixing it.
        System.out.printf("Hello and welcome!");

        for (int i = 1; i <= 5; i++) {
            //TIP Press <shortcut actionId="Debug"/> to start debugging your
            // code. We have set one <icon src="AllIcons.Debugger.Db_set_breakpoint"/>
            // breakpoint
            // for you, but you can always add more by pressing <shortcut
            // actionId="ToggleLineBreakpoint"/>.
            System.out.println("i = " + i);
        }
    }
}
```

Document.java

```
package com.cognizant.factory;

public interface Document {
    void open();
}
```

WordDocument.java

```
package com.cognizant.factory;

public class WordDocument implements Document {

    @Override
    public void open() {
        System.out.println("Opening Word Document...");
    }
}
```

PdfDocument.java

```
package com.cognizant.factory;

public class PdfDocument implements Document {
```

```
@Override
public void open() {
    System.out.println("Opening PDF Document...");
}
}
```

ExcelDocument.java

```
package com.cognizant.factory;

public class ExcelDocument implements Document {

    @Override
    public void open() {
        System.out.println("Opening Excel Document...");
    }
}
```

DocumentFactory.java

```
package com.cognizant.factory;

public abstract class DocumentFactory {

    public abstract Document createDocument();
}
```

WordDocumentfactory.java

```
package com.cognizant.factory;

public class WordDocumentFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new WordDocument();
    }
}
```

PDFDocumentfactory.java

```
package com.cognizant.factory;

public class PdfDocumentFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new PdfDocument();
    }
}
```

ExcelDocumentFactory.java

```
package com.cognizant.factory;

public class ExcelDocumentFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new ExcelDocument();
    }
}
```

factoryMethodTest.java

```
package com.cognizant.factory;

public class FactoryMethodTest {

    public static void main(String[] args) {

        DocumentFactory wordFactory = new WordDocumentFactory();
        Document word = wordFactory.createDocument();
        word.open();

        DocumentFactory pdfFactory = new PdfDocumentFactory();
        Document pdf = pdfFactory.createDocument();
        pdf.open();

        DocumentFactory excelFactory = new ExcelDocumentFactory();
        Document excel = excelFactory.createDocument();
        excel.open();

    }
}
```

Output



The screenshot shows the 'Run' console of an IDE. At the top, there is a tab labeled 'FactoryMethodTest'. Below the tab, the console output is displayed in a monospaced font. The output shows the execution of the main method, which creates and opens Word, PDF, and Excel documents. The process ends with the message 'Process finished with exit code 0'.

```
Run  FactoryMethodTest x
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\
Opening Word Document...
Opening PDF Document...
Opening Excel Document...
Process finished with exit code 0
```